ANALYTIC PROPOSAL • TEXT-TO-SQL

Texa

- Hỏi đáp ngôn ngữ tự nhiên
- Mọi dữ liệu có thể đối chứng, kiểm soát
- Vẫn tận dụng sức mạnh phân tích của model

Text-to-SQL trên semantic catalog

TỔNG QUAN

Texa

THÔNG ĐIỆP XUYÊN SUỐT

Làm sao để người dùng hỏi dữ liệu bằng ngôn ngữ tự nhiên, trong khi cách hiểu câu hỏi, cách tính và các con số trả lời đều đúng ý hoặc ít nhất là có thể kiểm tra và làm cho đúng ý?

TEXA

Phần 1 - Vấn đề/Pain point của doanh nghiệp

Một câu hỏi ngắn, nhiều quyết định ẩn

"Doanh thu tháng trước của cửa hàng là bao nhiêu?"

- Doanh thu hàng hóa hay số tiền thanh toán?
- Cửa hàng nào? Tháng nào trong dữ liệu lịch sử?
- Theo tiền tệ nào? Có đủ dữ liệu để tính không?
- Làm sao kiểm tra con số trong câu trả lời?

Điểm cần nhớ: “Bắt đầu từ một nhu cầu rất quen thuộc: người làm kinh doanh muốn hỏi dữ liệu mà không phải tự viết SQL

Mục tiêu và câu hỏi nghiên cứu

Câu hỏi nghiên cứu	Điều cần đo
RQ1. Hiểu đúng và tính đúng trong phạm vi hỗ trợ đến mức nào?	**Metric**: Ý định, kết quả, coverage **Diễn giải:** • Hiểu đúng người dùng muốn hỏi gì • Con số trả về có khớp với đáp án chuẩn không • Trả lời được bao nhiêu câu trong số những câu đáng lẽ phải trả lời được
RQ2. Có kiểm soát được mơ hồ, thiếu dữ liệu và diễn giải thiếu căn cứ không?	**Metric**: Clarification/refusal, numerical correctness, faithfulness **Diễn giải:** • Biết hỏi lại khi câu hỏi chưa rõ, biết từ chối khi dữ liệu không có (nhưng không từ chối những câu trả lời được) • Con số trong câu trả lời có đúng không • Phần nhận xét có bám đúng dữ liệu, không tự suy diễn không
RQ3. Chi phí của các bước kiểm soát có chấp nhận được không?	**Metric**: Latency, tokens, chi phí **Diễn giải:** • Người dùng phải chờ bao lâu để có câu trả lời • Mỗi câu hỏi tốn bao nhiêu lượt gọi mô hình và bao nhiêu tiền

TEXA

Phần 2 - Literature Review/Previous Work

Một số công trình, nghiên cứu trước đó

Công trình	**Ý tưởng chính**	**Điểm mạnh**	Điểm có thể cải thiện	Bài học cho TEXA
NaLIR, 2014 [R1]	Máy cho người dùng xem nó hiểu câu hỏi thế nào, rồi để người dùng xác nhận hoặc sửa	Câu hỏi mơ hồ được làm rõ thay vì bị đoán bừa	Người dùng phải thao tác thêm; máy phân tích câu sai thì hỏi lại cũng sai	Khi chưa rõ thì hỏi lại, đừng đoán
RAT-SQL, 2020 [R2]	Huấn luyện mô hình để nối từ trong câu hỏi với đúng bảng, đúng cột trong dữ liệu	Hiểu tốt cả những bộ dữ liệu chưa từng thấy	Cần dữ liệu để huấn luyện; nối đúng cột nhưng vẫn có thể tính saichọn sai logic	Nối đúng cột chưa đủ, còn phải tính đúng
IRNet, 2019 [R3]	Mô hình viết ra một “bản nháp” có cấu trúc trước, sau đó mới chuyển thành SQL	Tách việc hiểu ý khỏi việc viết câu lệnh	Bản nháp tốt đến đâu còn phụ thuộc dữ liệu huấn luyện và cách nối cột	Dùng bước trung gian để dễ kiểm tra

Điểm cần nhớ: Các hướng này giúp LLM hiểu câu hỏi tốt hơn, nhưng chưa bảo đảm hệ thống dùng đúng cách tính mà doanh nghiệp muốn. Ví dụ: “doanh thu” có trừ đơn hoàn hay không.

Một số công trình, nghiên cứu trước đó

Công trình	**Ý tưởng chính**	**Điểm mạnh**	Điểm có thể cải thiện	Bài học cho TEXA
DAIL-SQL (2024) [R4]	Không huấn luyện mô hình mới. Chỉ đưa cho LLM mô tả bảng dữ liệu kèm vài câu hỏi mẫu đã có đáp án, rồi để nó tự viết SQL	Dùng ngay được LLM có sẵn, dễ thay đổi, tiết kiệm chi phí gọi mô hình	Kết quả thay đổi nhiều tùy vào cách đưa gợi ý. LLM vẫn có thể hiểu sai nghiệp vụ	Cần chọn kỹ thông tin đưa cho mô hìnhoán
DIN-SQL (2023) [R5]	Chia việc thành nhiều bước: tìm đúng bảng/cột → chia nhỏ câu hỏi → viết SQL → tự kiểm tra và sửa	Mỗi loại lỗi được xử lý ở một bước riêng	Gọi mô hình nhiều lần nên chậm và tốn hơn. Bước đầu sai thì các bước sau sai theo	Chia bước có ích, nhưng thêm bước không có nghĩa là đúng hơn

Điểm cần nhớ: Đưa đúng thông tin và chia việc hợp lý đều giúp ích. Tuy vậy, không thể mặc định rằng cứ thêm bước hoặc cho mô hình tự sửa là kết quả sẽ đúng hơn. Điều này phải được đo và so sánh.

Một số công trình, nghiên cứu trước đó

Semantic Layer / MetricFlow [R7, R8] từng thiết kế workflow:



- **Điểm mạnh:** Cách tính được chốt một lần, mọi câu hỏi dùng chung, và có thể truy ngược mỗi con số được tính ra sao.
- **Hạn chế:** Phải tốn công định nghĩa và cập nhật. Không trả lời được những gì nằm ngoài định nghĩa. Bản thân định nghĩa vẫn có thể sai.
- **Bằng chứng:** Đây là thiết kế đang dùng trong công nghiệp. Chưa có thí nghiệm nào chứng minh nó giúp TEXA chính xác hơn, và TEXA sẽ kiểm chứng điều này.

Tổng hợp hạn chế → Nhiệm vụ của TEXA

Ví dụ: Người dùng hỏi doanh thu năm 2018, máy tạo yêu cầu “doanh thu + tên db + 2019”. Yêu cầu này hoàn toàn hợp lệ về định dạng, nhưng trả lời sai câu hỏi.

Kỹ thuật	Giúp được gì?	Nhưng vẫn có thể sai ở đâu
Nối từ ngữ với dữ liệu (schema / semantic linking)	Hiểu “doanh thu” là cột nào, khái niệm nào	Nối đúng từ nhưng vẫn hiểu sai ý cả câu
Bản nháp trung gian (intermediate representation)	Các lựa chọn của máy hiện ra rõ ràng, dễ kiểm tra	Hiện rõ không có nghĩa là các lựa chọn đều đúng
Chặn câu lệnh sai cú pháp (PICARD [R6])	Loại câu lệnh viết sai ngay trong lúc máy đang viết	Câu lệnh đúng cú pháp vẫn có thể tính sai nghiệp vụ
Bắt đầu ra theo mẫu, rồi kiểm tra (structured JSON + validation)	Chỉ chấp nhận chỉ số, nguồn và tổ hợp có thật trong hệ thống	Vẫn có thể chọn nhầm nguồn hoặc nhầm năm (2019 thay vì 2018)
Chỉ đưa thông tin liên quan (context / example selection)	Mô hình bớt bị nhiễu bởi thông tin thừa	Có thể vô tình lược bỏ đúng thông tin cần dùng

Điểm cần nhớ: TEXA kiểm tra riêng ba mức: đúng định dạng → đúng ý và đúng số → diễn giải đúng. Qua được mức đầu không chứng minh hai mức sau.

Best practices và các kiểm chứng, đánh giá

Best practice được chọn	Học từ đâu	Điều phải kiểm chứng
1. Gắn câu hỏi với định nghĩa có sẵn “doanh thu” phải ứng với một công thức đã được chốt	Nối từ ngữ với dữ liệu; semantic layer	Có chọn đúng chỉ số và đúng nguồn dữ liệu không?
2. Viết “bản nháp” trước, chưa rõ thì hỏi lại **Không đoán khi câu hỏi mơ hồ**	NaLIR; IRNet	Có bớt hiểu sai mà không làm người dùng phiền vì hỏi quá nhiều không?
3. Kiểm soát việc chạy truy vấn, ghi lại nguồn của từng con số	Quyết định thiết kế của nhóm	Con số có đúng không? Lời giải thích có bám đúng số không?
4. Đo riêng từng bước, kèm thời gian và chi phí	Các tổng hợp về đánh giá [R12]	Chất lượng tăng thêm có đáng với thời gian và tiền bỏ ra không?

Khái quát flow: Hiểu câu hỏi → Kiểm tra dữ liệu có đủ không → Chạy truy vấn an toàn → Trả lời dựa trên kết quả

TEXA

Phần 3 - Available Datasets

Vì sao chọn Cosmetics và Olist?

Hai bộ dữ liệu bán hàng thật, công khai, bổ sung cho nhau: bộ này có thứ bộ kia thiếu. Nhờ vậy, TEXA phải biết khi nào trả lời được và khi nào phải nói “không có dữ liệu”.

Tiêu chí	REES46 Cosmetics [D1]	Brazilian E-Commerce / Olist [D2]
Là gì	Nhật ký hành vi của một shop mỹ phẩm: xem → thêm giỏ → mua	Đơn hàng của một sàn thương mại điện tử: sản phẩm, thanh toán, giao hàng, đánh giá
Hỏi được	“Bao nhiêu % lượt xem dẫn đến mua?” “Thương hiệu nào bị bỏ giỏ nhiều nhất?”	“Bang nào giao hàng chậm nhất?” “Danh mục nào bị đánh giá thấp?”
Không hỏi được	Giao hàng, đánh giá, khách ở đâu	Khách đã xem hay bỏ giỏ gì trước khi mua
Quy mô	~20 triệu lượt hành vi (2019–2020, USD)	~100 nghìn đơn hàng (2016–2018, BRL)

Vì sao cần cả hai:

1. **Phủ được nhiều loại câu hỏi hơn**. Cùng nhau, hai bộ bao được cả hành vi mua sắm lẫn vận hành đơn hàng (RQ1).
2. **Buộc hệ thống biết từ chối đúng lúc.** Hỏi về đánh giá trên Cosmetics phải được trả lời là “không có dữ liệu”, không được bịa ra số (RQ2).
3. **Có sẵn nhiều bẫy tính toán giống thực tế**. Khác tiền tệ, khác thời kỳ. Một đơn có nhiều sản phẩm, nhiều lần thanh toán, nên dễ đếm trùng. Đây là chỗ để kiểm tra xem hệ thống có tính đúng không (RQ1, RQ2).

ETL dự kiến và hợp đồng dữ liệu

Chuẩn bị dữ liệu: từ file gốc đến bộ dữ liệu dùng để đánh giá

1 Dữ liệu gốc

CSV nguồn

Cosmetics
1 file nhật ký hành vi

Olist
8 file: đơn hàng, sản phẩm, thanh toán, đánh giá…

2 Lưu nguyên bản

Raw

- Giữ nguyên, không sửa gì
- Dùng để đối chiếu khi nghi ngờ số liệu

3 Làm sạch

Staging

- Chuẩn kiểu: ngày, số, tiền
- Bỏ bản ghi trùng (chỉ khi có căn cứ)
- Ghi lại dòng lỗi, không âm thầm xóa

4 Gộp về một bảng

Event mart thống nhất

- Một khuôn chung cho 2 nguồn
- Mỗi dòng ghi rõ: nguồn, thời gian, khách, sản phẩm, giá
- Chống đếm trùng trước khi tính chỉ số

Kiểm tra chất lượng

quality checks

- Tổng số khớp với bản gốc
- Không trùng, không thiếu khóa
- Ngày tháng hợp lý

Phiếu thông tin cho TEXA

metadata

- Dữ liệu từ ngày nào đến ngày nào
- Có sự kiện, cột nào; thiếu bao nhiêu
- Tiền tệ gì

Bản chụp cố định

snapshot dùng để đánh giá

- Khóa lại một phiên bản dữ liệu
- Mọi lần chấm điểm dùng chung
- **Kết quả so sánh được với nhau**

TEXA

Phần 4 - Proposal Methods + Evaluation Methods

Áp dụng best practice thành kiến trúc đề xuất

1 Hiểu câu hỏi

Chọn phần liên quan — LLM · tùy chọn
shortlist

Chỉ dùng khi catalog lớn: lọc bớt chỉ số không liên quan

Hiểu ý câu hỏi — LLM
interpret · gọi model lần 1

Câu hỏi + catalog → bản nháp: chỉ số, chia nhóm, thời gian

Hỏi lại nếu chưa rõ
clarify

Mơ hồ → hỏi lại bằng nút bấm
Đủ rõ → tạo **QuerySpec**

2 Kiểm tra dữ liệu đủ không

Kiểm tra hợp lệ
validator

Chỉ số, nguồn, bộ lọc có thật trong catalog? Đúng giới hạn?

Kiểm tra dữ liệu đủ không
availability check

Nguồn không ghi nhận → từ chối hoặc nêu rõ, không báo số 0

3 Chạy truy vấn an toàn

Tạo câu lệnh SQL
compiler

Dùng công thức định nghĩa sẵn, tên cột chỉ lấy từ catalog

Chạy truy vấn
execute

Tài khoản chỉ đọc, giới hạn số dòng trả về

Tính các con số
facts

Code tính tổng, tỷ lệ, xu hướng → danh sách số có nguồn gốc

4 Trả lời dựa trên kết quả

Viết câu trả lời — LLM
insight · gọi model lần 2

Chỉ trích số qua mã [[f_…]], không tự gõ con số nào

Thay số thật & lọc
server

Thay mã bằng số thật; câu dẫn số không có nguồn bị bỏ

Câu trả lời

Số + nhận xét + cảnh báo
Kèm SQL và chi tiết để kiểm tra

Catalog ngữ nghĩa · nền tảng cho bước 1–3

Định nghĩa chỉ số và chiều phân tích đã duyệt · mỗi nguồn có sự kiện gì · dữ liệu đến ngày nào · tiền tệ

Gọi mô hình ngôn ngữ (LLM)

Code kiểm soát, không dùng mô hình

Texa: Sản phẩm cuối dự kiến

Chuẩn bị dữ liệu và tri thức

CSV Cosmetics / Olist > Airflow: raw > staging > event mart > PostgreSQL warehouse
+ Metadata: thời gian, vocabulary, coverage, currency + Semantic catalog: metrics, dimensions, mappings, glossary

Tóm tắt quy trình Xử lý câu hỏi từ sơ đồ kiến trúc:

1. Câu hỏi + ngữ cảnh > LLM hiểu ý định thành candidate
2. Python chuẩn hóa, xác định thông tin còn thiếu > Cần hỏi lại? Nếu có, người dùng xác nhận
3. QuerySpec cuối + validation + availability > Nếu không hỗ trợ: giải thích giới hạn / từ chối
4. Compiler thành SQL có tham số > Thực thi chỉ đọc + giới hạn tài nguyên
5. Python: bảng kết quả, facts, phép tính > LLM viết nhận xét từ facts / tham chiếu
6. Server kiểm tra và điền số > Câu trả lời + định nghĩa + SQL + nguồn số liệu

Kỹ thuật chấm đề xuất: so đầu ra với đáp án, từng trường

Câu hỏi
mỗi câu kèm đáp án kỳ vọng viết trước

→

Hệ thống chạy
đủ cả luồng: hiểu câu hỏi, kiểm tra, hỏi lại

→

Đầu ra hệ thống
status · QuerySpec cuối form hỏi lại · nguồn của số

→

So với đáp án
chỉ những trường đáp án có khai báo

→

Kết quả
Lệch 0 trường → **ĐẠT**
Lệch ≥ 1 trường → **TRƯỢT**

1 So trạng thái trước

trạng thái hệ thống = trạng thái kỳ vọng?

- `answered` · trả lời
- `needs_clarification` · hỏi lại
- `refused` · từ chối
- `conversation` · trò chuyện
- `discussed` · bàn trên kết quả cũ

Sai trạng thái → **TRƯỢT ngay**, không chấm tiếp

đúng

2 Đúng trạng thái → chấm chi tiết theo từng tình huống

`needs_clarification`

Các trường kỳ vọng phải hỏi ⊆ các trường trong form hỏi lại

VD: “Số người mua theo tuần” → phải hỏi cả source lẫn time_range

`answered`

So QuerySpec cuối (bản đã qua validator, thực sự đem đi chạy)

Chỉ số	`metric`	khớp, hoặc thuộc các cách tính chấp nhận
Nhóm · nguồn · chỉ số	`dimensions · sources`	so như tập hợp, bỏ qua thứ tự
Bộ lọc	`filters`	cùng trường; giá trị bỏ qua hoa/thường
Thời gian	`time_range`	kind, days, start, end sau khi quy đổi

`refused · conversation`

Chỉ cần đúng trạng thái là đạt

Áp dụng cho mọi trạng thái: đáp án có thể yêu cầu câu trả lời phải có số do hệ thống tính, hoặc cấm số tự tính và tỷ giá người dùng chưa đưa

Trường nào không khai báo trong đáp án thì không chấm

1 ví dụ evaluation

Câu hỏi: “Doanh thu theo danh mục của Cosmetics tháng 12/2019.”

Full QuerySpecs
(Hệ thống tạo ra)

```
{
  "source_ids": ["cosmetics_shop"],
  "contract_version": 1,
  "shape": "breakdown",
  "recipe": "revenue",
  "selection": ["item_category"],
  "filters": [],
  "time_range": {
    "kind": "between",
    "start": "2019-12-01",
    "end": "2019-12-31"
  },
  "limit": 1000
}
```

ExpectedSpecs
(Đáp án kỳ vọng cần có:)

```
{
  "status": "answered",
  "source_ids": ["cosmetics_shop"],
  "shape": "breakdown",
  "recipe": "revenue",
  "selection": ["item_category"],
  "filters": [],
  "time_range": {
    "kind": "between",
    "start": "2019-12-01",
    "end": "2019-12-31"
  }
}
```

Ví dụ: từ câu hỏi đến QuerySpec, rồi so với đáp án

Câu hỏi: *“Doanh thu theo danh mục của Cosmetics tháng 12/2019”*

Các bước hệ thống thực hiện

1 Hiểu câu hỏi — LLM

Bản nháp: doanh thu · theo danh mục · Cosmetics · tháng 12/2019

2 Đối chiếu catalog, hỏi lại nếu cần — code

“doanh thu” → chỉ số revenue đã được duyệt
Đủ rõ → không cần hỏi lại → tạo **QuerySpec**

3 Kiểm tra dữ liệu đủ không — code

Chỉ số và chiều có trong catalog ✓
Cosmetics có ghi nhận sự kiện mua hàng ✓

4 Chạy truy vấn và trả lời — code · LLM

SQL tạo từ công thức có sẵn, chạy chỉ đọc
Model viết lời, mọi con số do code tính

So QuerySpec với đáp án kỳ vọng

Trường	Đáp án kỳ vọng	Hệ thống tạo ra	Khớp
Nguồn	`["cosmetics_shop"]`	`["cosmetics_shop"]`	✓
Dạng câu hỏi	`breakdown`	`breakdown`	✓
Chỉ số	`revenue`	`revenue`	✓
Chia theo	`["item_category"]`	`["item_category"]`	✓
Bộ lọc	`[]`	`[]`	✓
Thời gian	`2019-12-01→12-31`	`2019-12-01→12-31`	✓

Phiên bản catalog, giới hạn số dòng: chi tiết kỹ thuật, không chấm

✓ **ĐẠT: 6/6 trường khớp → hệ thống hiểu đúng câu hỏi**

Nếu hệ thống ghi **tháng 11** → lệch “Thời gian” → **TRƯỢT**
và biết ngay lỗi nằm ở **bước 1 (hiểu câu hỏi)**, không phải phần tính

QuerySpec là điểm nối giữa phần hiểu (model) và phần tính (code): lệch QuerySpec → lỗi hiểu · QuerySpec đúng mà số sai → lỗi tính

Đóng góp dự kiến, trade-offs và tiêu chí quyết định

Đóng góp dự kiến

- Thiết kế tích hợp LLM, metric catalog, compiler và câu trả lời truy vết được.
- Data/metric contracts cho hai nguồn e-commerce khác nhau.
- Protocol đánh giá tách ý định, kết quả, diễn giải và chi phí.

Trade-offs: linh hoạt ↔ phạm vi catalog; kiểm soát ↔ công sức mô hình hóa; hỏi lại ↔ trải nghiệm; nhiều bước ↔ latency.

Quyết định dựa trên: baseline + ablation + held-out evaluation, chưa dựa trên thành tích giả định.

T E X A

Cảm ơn đã lắng nghe

Hỏi & đáp